

UNIVERSIDADE FEDERAL DE SANTA CATARINA - UFSC  
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA - INE  
INE5413 - GRAFOS

## **RELATÓRIO DA ATIVIDADE 1**

Igor Velmud Bandero

FLORIANÓPOLIS - SC  
Maio de 2026

## Introdução

Este relatório detalha a solução dos exercícios da Atividade 1. A implementação dos exercícios foi feita em C++, visando o controle sobre a alocação dinâmica de memória e a facilidade da orientação a objetos.

Para a execução dos exercícios, é necessário realizar a compilação e as chamadas via terminal. A compilação dos arquivos pode ser feita utilizando um compilador como o G++. Para compilar o “A1\_1.cpp”, por exemplo, deve-se utilizar a instrução “g++ A1\_1.cpp -o A1\_1” no terminal, que gerará um executável.

Para o A1\_1.cpp, uma vez gerado o executável, ele deverá ser invocado passando o caminho do arquivo de entrada que contém a estrutura do grafo (ex: “./A1\_1 fln\_pequena.net”), onde o programa fará a leitura da seção de vértices, mapeará os rótulos e, em seguida, construirá a lista de adjacências especificadas no arquivo “fln\_pequena.net”.

## Exercício 1 - Representação

No exercício 1, a estrutura central é a do Grafo, composta pelo número de vértices e arestas, por um vetor com as listas de adjacências e outro vetor de vértices. A escolha justifica-se pela economia de memória e agilidade na inserção. A estrutura Aresta encapsula o destino e o peso, enquanto a estrutura Vértice armazena o rótulo e índice, permitindo que o vetor Vértice ofereça acesso direto  $O(1)$  aos dados.

## Exercício 2 - Buscas

No exercício 2, utilizou-se a biblioteca *queue* (fila) para gerenciar a fila da busca, garantindo a exploração por níveis. Os vetores para o nível e para o estado “visitado” foram usados para controlar a profundidade e evitar redundâncias na busca. Para a saída, o vetor “níveis\_encontrados” organiza os nós por camada.

## Exercício 3 - Ciclo Euleriano

No exercício 3, utilizou-se a biblioteca *stack* (pilha) para implementar o algoritmo de Hierholzer, permitindo o processo de tentativa e erro necessário para construir o ciclo. A estrutura “adj\_temp” foi usada para gerenciar a remoção de arestas, utilizando ponteiros booleanos para sincronizar a marcação de arestas não-dirigidas.

## Exercício 4 - Algoritmo de Bellman-Ford ou de Dijkstra

No exercício 4, a estrutura central é a *priority\_queue* (fila de prioridades) que organiza os dados para que o algoritmo de Dijkstra encontre o caminho mais curto. Utilizou-se o tipo *pair<double, int>* para associar distâncias a vértices e um vetor

“antecessor” para o rastreamento do caminho. A biblioteca *stack* foi aplicada para inverter os antecessores e exibir a rota na ordem correta.

### **Exercício 5 - Algoritmo de Floyd-Warshall**

No exercício 5, a estrutura central é uma matriz de distâncias implementada com um vetor de vetor. Essa escolha permite que o algoritmo de Floyd-Warshall acesse e atualize as distâncias entre todos os pares de vértices em tempo constante  $O(1)$ . A matriz é inicializada com o valor INF (infinito) e preenchida com os pesos das arestas via métodos desenvolvidos no Exercício 1.